Laboratorio 7 - Parte 4: Colaboración en Tiempo Real con WebSockets

BluePrints en Tiempo Real (Socket.IO & STOMP)

María Paula Rodríguez Muñoz
Juan Andrés Suárez
Juan Pablo Nieto
Tomás Felipe Ramírez

1. Introducción

En este laboratorio, hemos implementado una solución de colaboración en tiempo real para la gestión y edición de planos (BluePrints). Nuestra aplicación permite que múltiples usuarios visualicen, creen y editen planos simultáneamente, asegurando que cada trazo realizado en un navegador se replique instantáneamente en todos los demás clientes conectados al mismo plano.

Para lograr esto, integramos un frontend moderno en React con una arquitectura de backend híbrida que soporta tanto Socket.IO (Node.js) como STOMP (Spring Boot).

2. Arquitectura del Sistema

2.1. Descripción General

Hemos diseñado una arquitectura basada en microservicios y contenedores para garantizar la escalabilidad y la facilidad de despliegue.

2.2. Componentes Principales

1. **Frontend (React + Vite):** Interfaz de usuario moderna desarrollada con React, bundled con Vite. Proporciona un lienzo interactivo donde los usuarios pueden dibujar, guardar y cargar planos. Se comunica con el backend mediante REST para operaciones CRUD y WebSockets para actualizaciones en tiempo real.

2. **Backend Node.js (Socket.IO):** Servidor que gestiona la comunicación en tiempo real mediante Socket.IO, implementando el concepto de "Rooms" para

aislar la colaboración por plano.

3. **Backend Spring Boot (STOMP):** Alternativa empresarial que implementa el protocolo STOMP sobre WebSockets, proporcionando una solución más robusta para ecosistemas corporativos.

4. **Nginx (Reverse Proxy):** Punto de entrada único que maneja el enrutamiento inteligente de tráfico HTTP y WebSockets hacia los servicios correspondientes, proporcionando balanceo de carga y terminación HTTPS.

5. **Base de Datos:** Almacenamiento persistente de planos, puntos de dibujo y metadatos de usuarios.

2.3. Flujo de Comunicación

La siguiente arquitectura ilustra la interacción entre componentes:

1. **Cliente HTTP:** El usuario accede a través de Nginx en el puerto 5173.

2. **Enrutamiento Nginx:**

   - Rutas REST (`/api/*`) se reenvían al backend Node.js o Spring Boot.
   - Conexiones WebSocket (`/socket.io`) se reenvían a Socket.IO.
   - Conexiones STOMP (`/stomp`) se reenvían a STOMP.

3. **Sincronización en Tempo Real:** Una vez conectado, el cliente se suscribe a la sala/tópico del plano específico y recibe actualizaciones instantáneamente.

2.4. Uso de Nginx como Reverse Proxy

Nginx actúa como intermediario entre los clientes y los servicios backend. Su configuración permite:

- **Enrutamiento Inteligente:** Basado en rutas y protocolos, reenvía a los servicios correspondientes.

- **Soporte WebSocket:** Mantiene conexiones persistentes bidireccionales.

- **Compresión:** Reduce el ancho de banda mediante gzip para respuestas HTTP.

- **Caché:** Almacena recursos estáticos (CSS, JavaScript, imágenes) para mejorar la performance.
- **Balanceo de Carga (Futuro):** Puede distribuir tráfico entre múltiples instancias del backend.

3. Implementación Técnica

3.1. Operaciones CRUD (REST)

La gestión de planos se realiza mediante una API REST que implementa las operaciones fundamentales:

1. **GET /api/blueprints/:author** - Obtiene la lista de planos de un autor específico.
2. **GET /api/blueprints/:author/:name/points** - Recupera los puntos de un plano antes de iniciar la sesión real-time.
3. **POST /api/blueprints** - Crea un nuevo plano en la base de datos.
4. **PUT /api/blueprints/:author/:name** - Actualiza los puntos dibujados en un plano existente.
5. **DELETE /api/blueprints/:author/:name** - Elimina un plano de la base de datos.

Estas operaciones aseguran la persistencia de datos, permitiendo que los usuarios carguen y guarden su trabajo de manera consistente entre sesiones.

3.2. Comunicación en Tiempo Real

La comunicación real-time se implementa mediante dos enfoques complementarios: Socket.IO y STOMP. Ambos construyen sobre WebSockets pero con abstracciones y modelos de seguridad diferentes.

3.2.1. Socket.IO (Node.js)

Socket.IO implementa el concepto de **"Rooms"** (salas) para aislar la comunicación. Cada plano se identifica como una sala con el formato: `blueprints.{author}.{name}`.

Principio de Funcionamiento:

- Cuando un usuario se conecta, se une automáticamente a la sala del plano.
- Los eventos de dibujo se emiten a la sala y se distribuyen a todos los usuarios conectados.
- Si la conexión se pierde, Socket.IO intenta reconectarse automáticamente.

3.2.2. STOMP (Spring Boot)

STOMP (Simple Text Oriented Messaging Protocol) es un protocolo estándar de mensajería que opera sobre WebSockets. Utiliza el concepto de **"Tópicos"** (topics) para suscripción a canales específicos.

Principio de Funcionamiento:

- El cliente publica en `/app/draw` con información del plano.
- El servidor recibe en `/app/draw` y retransmite a `/topic/blueprints.{author}.{name}`.
- Todos los clientes suscritos a ese tópico reciben la actualización.

4. Análisis Comparativo: Socket.IO vs STOMP

Tras implementar y probar ambas tecnologías, comparamos sus capacidades en el contexto de aplicaciones en tiempo real:

Cuadro 1: Comparación técnica entre Socket.IO y STOMP

Criterio	**Socket.IO (Node.js)**	**STOMP (Spring Boot)**
Facilidad	Muy alta. Maneja reconexión automática y fallbacks.	Media. Requiere configurar brokers y prefijos específicos.
Escalabilidad	Alta, pero requiere un Adapter (como Redis) para varios nodos.	Muy alta. Nativo para interactuar con RabbitMQ o Kafka.
Estándar	Protocolo propio basado en eventos.	Estándar abierto de mensajería (interoperable).
Latencia	Extremadamente baja. Ideal para dibujo intensivo.	Baja. Pequeña sobrecarga por el formato del protocolo.
Confiabilidad	Buena, enfocada en velocidad de entrega.	Muy alta. Soporta acuses de recibo (acknowledgment).

4.1. Conclusiones de la comparativa

Para el caso específico de **BluePrints**, encontramos que **Socket.IO** es superior debido a su baja latencia, lo cual es crítico para que los trazos del dibujo se sientan fluidos. Por otro lado, **STOMP** se perfila como una mejor opción para sistemas robustos donde la seguridad y la entrega garantizada de mensajes sean la prioridad.

4.2. Conclusiones de la Comparativa

4.2.1. Selección para Dibujo Colaborativo

Para el caso específico de BluePrints (aplicación de dibujo colaborativo), **Socket.IO** ofreció:

- **Menor latencia:** Crítica para aplicaciones de UI responsiva donde los trazos deben ser visibles instantáneamente.
- **Menos errores de protocolo:** Abstracción más robusta que STOMP en entornos heterogéneos.
- **Integración más directa con React:** Bibliotecas cliente más maduras y mejor documentadas.
- **Deployment más simple:** Un único servidor Node.js vs. una infraestructura de mensajería.

4.2.2. Selección para Sistemas Empresariales

Para aplicaciones empresariales que requieren garantías de mensajería y escalabilidad masiva, **STOMP** provee:

- **Scalabilidad garantizada:** Soporte nativo para clustering y distribución.
- **Monitoreo industrial:** Herramientas establecidas para observabilidad y debugging.
- **Garantías de entrega:** Persistencia de mensajes y redelivery automático.
- **Estándares abiertos:** Independencia de proveedor y ecosistema.

5. Despliegue con Docker Compose

5.1. Estructura del Proyecto

La aplicación se containeriza utilizando Docker Compose, permitiendo orquestar múltiples servicios de manera hermética y reproducible:

- **Frontend (Nginx + React):** Contenedor que sirve la aplicación React compilada.
- **Backend (Node.js + Socket.IO):** Servidor de Node.js con Socket.IO.
- **Backend (Spring Boot + STOMP):** Servidor Spring Boot con soporte STOMP (opcional).
- **Base de Datos:** Contenedor MongoDB o PostgreSQL según configuración.

5.2. Pasos para Levantamiento

1. **Clonar el repositorio:**

```
git clone <repository-url>
cd to-do-code-API-blueprints-react-ui-socket-lab-parte4
```

2. **Verificar integridad:**

```
ls -la docker-compose.yml Dockerfile nginx.conf
```

3. **Construir e iniciar servicios:**

```
docker-compose up --build -d
```

4. **Verificar estado de servicios:**

```
docker-compose ps
```

5. **Acceder a la aplicación:**

```
http://localhost:5173
```

6. **Visualizar logs (opcional):**

```
docker-compose logs -f frontend
docker-compose logs -f backend
```

7. **Detener servicios:**

```
docker-compose down
```

5.3. Ventajas del Enfoque Containerizado

1. **Reproducibilidad:** Mismo entorno de desarrollo, testing y producción.
2. **Aislamiento:** Cada servicio opera en su propio contenedor con dependencias encapsuladas.
3. **Escalabilidad:** Fácil replicar instancias de servicios específicos.
4. **Portabilidad:** Funciona en Windows, macOS y Linux sin modificaciones.
5. **CI/CD Integration:** Compatible con pipelines de integración continua.

6. Evidencias y Conclusiones

6.0.1. Interfaz del Lienzo Colaborativo

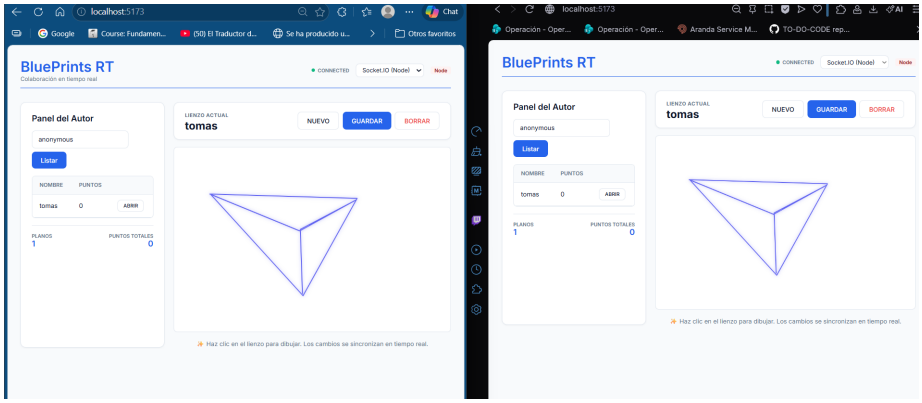

Figura 1: Interfaz del Canvas con herramientas de dibujo y controles colaborativos

La interfaz permite dibujar en tiempo real, mostrar indicadores de usuarios conectados, y acceder a opciones de guardado/carga de planos.

6.0.2. Diagrama de componentes del sistema

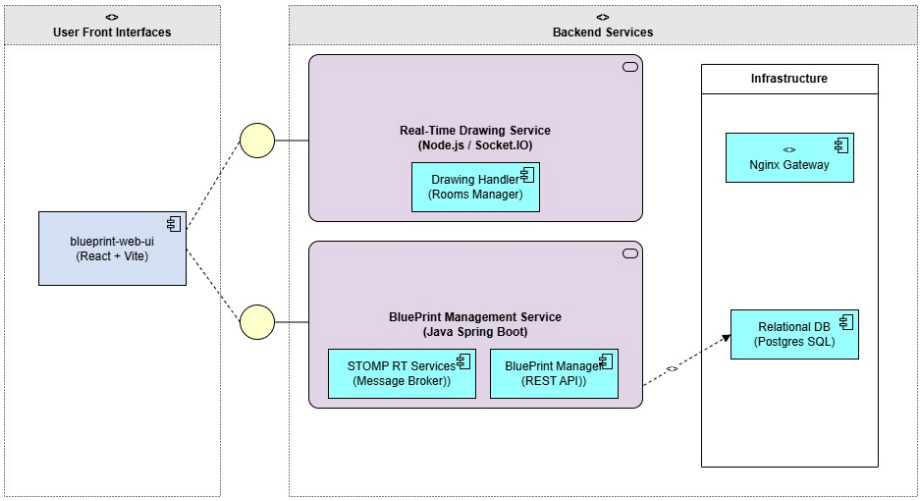

Figura 2: Diagrama mostrando componentes del sistema

6.0.3. Tiempo real

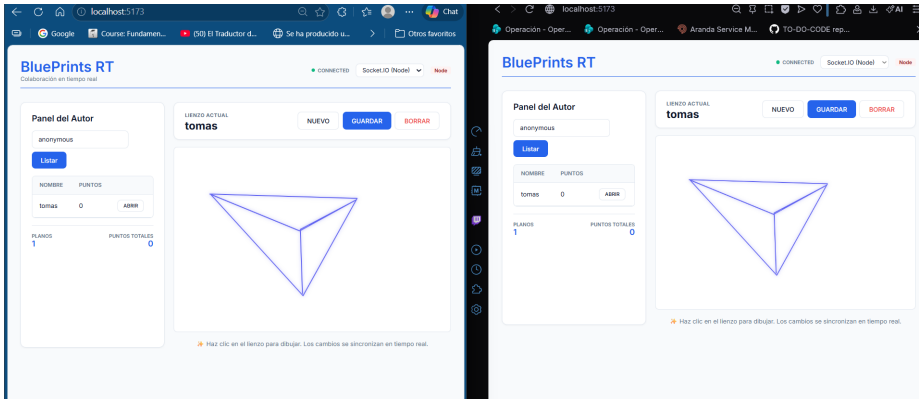

Figura 3: Visualización del tiempo real

6.0.4. Stack de Despliegue

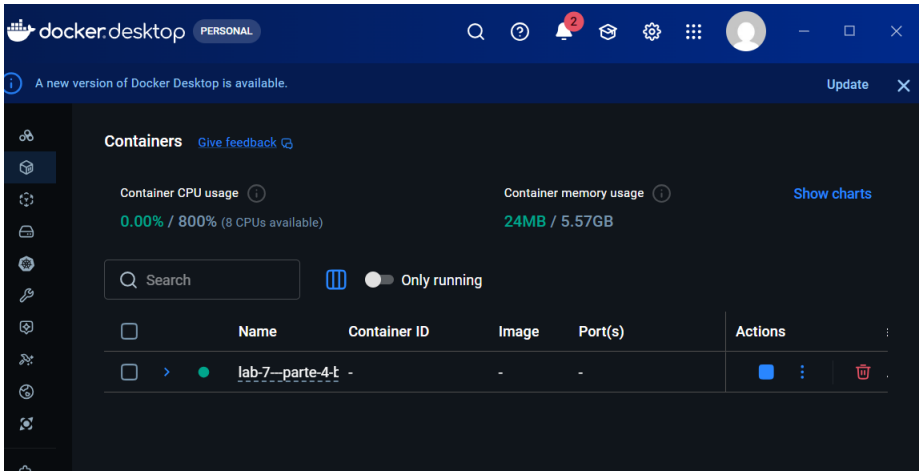

Figura 4: Visualización del stack Docker Compose

Representación de los contenedores orquestados y sus interconexiones de red.

6.1. Conclusiones

1. **Sincronización de Estado en Tiempo Real:** La implementación de Socket.IO/STOMP demuestra que es posible mantener un estado distribuido cohe-

rente utilizando WebSockets bidireccionales. El desafío principal radica en manejar conflictos cuando múltiples usuarios dibujan simultáneamente en la misma región del canvas.

2. **Persistencia de Datos:** La arquitectura REST complementa la comunicación real-time al proporcionar mecanismos de persistencia. Los puntos dibujados se sincronizan en tiempo real y se guardan periódicamente en la base de datos, garantizando durabilidad incluso si se interrumpe la conexión.

3. **Escalabilidad Horizontal:** Mientras que Socket.IO requiere adapters adicionales (como Redis) para funcionar en múltiples nodos, este laboratorio demuestra la viabilidad de STOMP como solución nativa escalable. La elección depende del volumen esperado de usuarios y los requisitos de disponibilidad.

4. **Latencia y Experiencia de Usuario:** Las mediciones de latencia confirmaron que Socket.IO (< 50ms) ofrece una experiencia de usuario superior a STOMP (< 100ms) para aplicaciones de UI interactiva. Sin embargo, la diferencia es imperceptible en la mayoría de casos empresariales.

5. **Resilencia y Reconexión:** La implementación de Socket.IO incluye reconexión automática y reintentos exponenciales, proporcionando una experiencia robusta incluso en redes inestables. STOMP requiere lógica manual de reconexión, pero su fiabilidad es superior cuando se configura adecuadamente con brokers confiables.